

YaPcore Admin Dashboard

Browser-based **admin control panel** for headless hosts and remote operators. Set up the network, configure YaP Link, manage plugins, and monitor health — no SSH required for day-to-day ops.

Controls the **YaPcore chassis** in front of **YaP-Folia** (game child JVM). Build with `./scripts/build-yap-folia.sh`.

Modern **sidebar shell** (Overview · Server · People · Content · Gameplay) with dark theme, page search, stat cards, and full plugin config editors where the backend supports it.

Colored text (ranks, MOTD, tab list, NPC names, chat format) uses **clickable Minecraft color swatches**, a custom hex picker, and bold/italic controls — plus a live preview. Hex is stored as `&#rrggbb`.

Enable

Default on. Config (`config/server.properties`):

```
web-dashboard-enabled=true
web-dashboard-port=8080
web-dashboard-bind=0.0.0.0
web-dashboard-token=
web-dashboard-localhost-only=false
ops=
auto-op=false
```

Empty `web-dashboard-token` → a random token is generated on first start and saved into the config file (also printed in the server log).

Open

```
http://127.0.0.1:8080/
```

Paste the token on the login screen (or set `Authorization: Bearer <token>`).

Quick access: type `dashboard` in the YaP Control Panel console or headless stdin — prints a one-click login URL and token. In the Swing GUI, use **Web Dashboard** (header) or **Connect → Open**.

Login links include `?token=...` so the browser signs in automatically (token is stripped from the address bar after load).

Navigation

Group	Tabs
Overview	Dashboard (status), Network setup, Connect
Server	Console, Server setup, YaP Link
People	Players, Access & ranks, Rank pack
Content	Plugins, Plugin settings, Modules, Packs, World, Regions, NPCs
Gameplay	Essentials, Vehicles, Pregen, Player data, Kits, Tebex, Chat, Tab list, MMO , Factions , Guilds , Games , Disasters , Stacker , Map, Guard, Protect, Discord

Static assets: `src/main/resources/web/` — `app-shell.js`, `app-core.js`, `app-*-panels.js`, `style.css`.

Admin tab (infrastructure)

`/api/admin` — operator setup without editing files by hand:

Section	What you configure
Dashboard access	Port, bind, localhost-only, enable/disable, rotate token
External access	Internet expose, domain, public Java/Bedrock/pack ports, DNS SRV hint
nginx	Localhost assist, stream/HTTP ports, domain, config dry-run
Proxy / authority	Velocity forwarding, Link embed mode, link home path

POST actions: `save-access`, `save-nginx`, `save-dashboard`, `save-proxy`, `rotate-token`, `nginx-dry-run`, `run-smoke`, `crashdump`.

All tabs & API routes

Tab	API	GET snapshot	POST actions (high level)
Dashboard	<code>/api/status</code>	running, heap, ticks, link process, network health	—
Connect	<code>/api/connect</code>	Java/Bedrock/crossplay join, pack URL	—
Console	<code>/api/console</code> , <code>/api/command</code>	log backlog	run command · SSE <code>/api/console/stream</code>
Server setup	<code>/api/config</code>	everyday server switches (name, who can join, RAM)	save config keys
YaP Link	<code>/api/link</code> , <code>/api/link/console</code>	proxy, backends, forced hosts, selector	start, stop, save-proxy, save-servers, command · Link SSE
Players	<code>/api/players</code>	online list, spawn, moderation flags	kick, ban, tempban, ipban, mute, warn, timeout, tp, <i>set-rank</i> , <i>promote</i> , <i>demote</i> , <i>eco</i> <i>give/take/set/reset*</i> , bal, history, check, banlist
Access & ranks	<code>/api/access</code>	ops, auto-op, default group, groups, tracks , server-context, permission catalog , group nodes	save-group-nodes, save-ops, save-auto-op, set-default-group, op, deop, set-group, promote/demote (+ track), user-perm / group-perm (+ duration, world, server), user-perm-unset / group-perm-unset , dump, reload, applypack
Rank pack	<code>/api/ranks</code>	pack applied, auto-apply	apply, force, reset-marker, status
Plugins	<code>/api/plugins</code>	jar list + compat matrix	install, remove
Plugin settings	<code>/api/plugin-config</code>	first-party YAML with plain-language titles + Yes/No	save + reload, reload
Modules	<code>/api/modules</code>	module jars	install, remove
Packs	<code>/api/packs</code>	resource packs, active set	setActive, add, remove, clear
World	<code>/api/world</code>	schematics, brush max, load/unload flags	load, unload, reload, schem-list, save-brush
Regions	<code>/api/regions</code>	region table (JSON), flag names	define (cuboid coords), flag-set , list

Tab	API	GET snapshot	POST actions (high level)
NPCs	/api/npcs	npc table, quest ids	create, remove, setquest, setdialogue, respawn, reload, info
Essentials	/api/essentials	features, MOTD, rules, spawn	reload, broadcast, save-motd, save-rules, set-feature
Vehicles	/api/vehicles	type list	spawn, shop, list, types, upgrades
Pregen	/api/pregen	job status	start, pause, resume, cancel
Player data	/api/playerdata	economy, auth, feature toggles	reload, save, set-feature
Kits	/api/kits	kits.yml definitions, items, armor slots	save-kit, delete-kit, clone-kit , give, grant, reload
Tebex store	/api/tebex	jar present, secret masked, buy command, proxy, package recipes	set-secret, save-settings , reload, info, forcecheck
Chat	/api/chat	channels, slow mode, filter, relay	reload, clearchat, save-settings
Tab list	/api/tab	header/footer/sidebar/bossbar	save-header/footer/sidebar/settings/bossbar, reload
Map	/api/map	map URL, tiles, worlds, render interval	reload, render, save-settings
Guard	/api/guard	check toggles, kick threshold, decay, alerts	reload, player-status, save-settings
Protect	/api/protect	logging, retention, status	reload, prune, lookup, lookup-radius, rollback, restore, save-settings
Stacker	/api/stacker	enabled, mob/item/spawner toggles, kill mode, live stats	save-settings , reload, status, stats
MMO	/api/mmo	skills, abilities, hiscores, boss kills, combat bar bindings	reload-abilities, reload-mmo
Factions	/api/factions	counts, preview, power/bank settings	save-settings , reload, setpower, setjoin, disband
Guilds	/api/guilds	counts, preview, level/bank settings	save-settings , reload, setlevel, disband
Games	/api/games	modes, arenas, match/reward toggles	save-settings , reload, list, forcestart
Disasters	/api/disasters	extremes, random schedule, volcano sites	save-settings , reload, start, stop, random, site-*

Legacy routes (superseded by UI tabs): /api/moderation → **Players**; /api/perms → **Access & ranks**.

Pack HTTP stays on **:8081**. Dashboard is a separate port (**:8080**).

Access & ranks

YaP **ops surface** for permissions — not a LuckPerms-web clone. Full operator control without /op and /yapperm by hand:

- **Minecraft OPs** — chip list, add/remove, persisted to server.properties
- **Auto-op** — toggle for first join
- **Default rank** — YaPPerms default group dropdown

- **Group cards** — click to edit tag, name color, and chat color with **color swatches + custom hex** (no & codes required), plus suffix, weight, and inheritance. Live chat preview updates as you pick colors.
- **Rank permissions** — catalog of player / staff / vanilla / Paper commands plus **nodes discovered from installed plugin.yml**; inherit · allow · deny; any custom / wildcard node (bulk add); saved with `save-group-nodes` (permanent + global via `yapperm editor-apply`)
- **Create rank with a pack** — empty / player / staff / admin, or copy perms from an existing rank (`template` , `cloneFrom` on `create-group`)
- **Apply pack / copy perms** onto an existing rank (`apply-template` , `clone-group`)
- **Promotion track** — ladder chips + track picker; **Promote / demote** steps one rank (`promote` / `demote` with optional `track`)
- **Context + temp grants** — Players pane grants/revokes a node with optional `duration` (`1h` , `1d` , `7d` , `30d` , or custom like `1d12h`), `world` , and `server` (wires to `yapperm user|group permission set|unset`)

POST `/api/access` {`"action": "save-group-nodes", "group": "vip", "allow": "yapessentials.fly, ...", "deny": "minecraft.command.op", "unset": "yapessentials.god"`} writes `plugins/YaPPerms/config.yml` (`starter-grants` + `editor-nodes`) and applies live via `yapperm editor-apply` . Refresh dumps live extras with `yapperm dump` → `editor-snapshot.yml` .

Timed / world-scoped example (does **not** go through the rank editor batch):

```
{ "action": "user-perm", "player": "Steve", "node": "yapessentials.fly", "value": "true", "duration": "7d", "world": "world" }
{ "action": "user-perm-unset", "player": "Steve", "node": "yapessentials.fly", "world": "world" }
{ "action": "group-perm", "group": "vip", "node": "yapmod.ban", "value": "true", "duration": "1d", "server": "lobby" }
{ "action": "promote", "player": "Steve", "track": "yap" }
```

MariaDB already stores `world` , `server_ctx` , and `expires_at` on user/group nodes; the dashboard now exposes those fields for ops.

Kits (`yap-playerdata`)

Gameplay → Kits builds claim kits in `plugins/YaPPlayerData/kits.yml` (same file `/createkit` uses):

- Cooldown, max uses, economy cost, first-join, console commands (`{player}`)
- Item rows: Bukkit material, amount, slot (inventory / armor / offhand), name, lore, enchantments (`sharpness:5`)
- Clone / delete; **Give now** (`kit give`) or **Grant** (`kit grant`) to a player
- Saves YAML then runs `yapdata reload` (YAML is kept if Folia is down)

GET/POST `/api/kits` — `save-kit` , `delete-kit` , `clone-kit` , `give` , `grant` , `reload` . Item lines in POST: `MATERIAL|amount|slot|name|lore|enchants` . Players still need `yapdata.kit.<id>` (or `yapdata.kit.*`) on Access & ranks.

`/createkit` Bukkit stacks stay readable; saving from the dashboard writes the material form (NBT beyond name/lore/enchants is dropped).

Tebex store

Gameplay → Tebex store wires the GPLv3 Folia plugin (`plugins/tebex.jar`):

- Status: jar present, secret configured (masked), `/buy` command, proxy mode
- **Save secret** → writes `plugins/Tebex/config.yml` and runs `tebex secret <key>`
- Buy command / proxy / verbose toggles → `tebex reload`
- Copy-ready package recipes (`{username}`) for VIP rank and kit unlocks
- Links to creator.tebex.io and Tebex Minecraft docs

GET/POST /api/tebex — set-secret , save-settings , reload , info , forcecheck . Full guide: [TEBEX.md](#).

Players

Staff moderation panel: **online** table plus **everyone who has ever joined** (username, nickname, UUID, last IP, all known IPs, first/last seen). Click a row to kick/ban/mute/IP-ban. IP bans use the stored last IP after they leave.

Economy on the same panel: amount field + **Give money** / **Take money** / **Set balance** / **Reset** / **Check balance** (eco ... / bal via console).

Requires YaP-Folia running + yap-moderation / yap-perms / yap-playerdata . Snapshot is refreshed with yapmod seen snapshot when you hit Refresh.

MMO (yap-skills , yap-abilities , yap-mmo-content)

Gameplay → MMO tab — read-only progression overview plus ability hotbar status:

- Skill/content jar presence, ability catalog count, boss/area counts
- Dual hotbar + ability book flags (including Shift+F)
- Online players with combat bar bindings (keys 4-9)
- Hiscore preview + boss kill totals (live when Folia is running)

Action	POST /api/mmo
Reload ability YAML	{"action":"reload-abilities"}
Reload MMO content	{"action":"reload-mmo"}

Live data uses console exports: yapmmo snapshot json , yapabilities snapshot json .

Factions (yap-factions)

Gameplay → Factions — counts + preview, YAML settings (power/bank/relations), admin console actions:

Action	POST /api/factions
Save settings + reload	{"action":"save-settings",...}
Reload	{"action":"reload"}
Set power	{"action":"setpower","faction":"Tag","power":"50","max":"60"}
Set join mode	{"action":"setjoin","faction":"Tag","mode":"invite"}
Force disband	{"action":"disband","faction":"Tag"}

Player create/join/claim stays in-game (/f ...).

Guilds (yap-guilds)

Gameplay → Guilds — counts + preview, level/bank settings, admin setlevel / force-disband via /yapguilds

Games (yap-games)

Gameplay → Games — mode/arena inventory, match toggles, ygames reload|list|forcestart .

Disasters (yap-disasters)

Gameplay → Disasters — extremes, random schedule, volcano sites. See /api/disasters POST actions in the table above.

Stacker (yap-stacker)

Gameplay → Stacker — enable toggles, kill mode, max stacks, live `/yapstacker status|stats`.

Action	POST /api/stacker
Save YAML + reload	<code>{"action":"save-settings",...}</code>
Reload	<code>{"action":"reload"}</code>
Stats	<code>{"action":"stats"}</code>

YAML-only by design (not dashboard gaps)

These ship as **Plugin settings** editors (or in-game hubs) on purpose — they do not need a dedicated interactive ops tab:

Plugin / area	Why YAML / elsewhere
LagGuard	Already on status metrics
YaPAdmin	In-game staff hub (/yapadmin)
gameplay-knobs	Tunables via Plugin settings
skills / combat / crafting / mechanics	MMO tab + YAML packs
floodgate / bedrock-ui / folia-bridge	Crossplay bridge config
placeholderapi / plugin-compat	Expansion / soft-dep config

NPCs (yap-npcs)

Dashboard drives the plugin directly:

- Create NPC at world coordinates (console: `npc create <id> at <world> <x> <y> <z> [yaw] [name]`)
- Edit display name, quest, dialogue; remove; respawn all; reload config
- GET `/api/npcs` returns structured `npcs[]` from `npc list json`

Regions (yap-regions)

- Define / redefine / remove admin cuboids from the UI
- Set WorldGuard-class flags (including item-drop/pickup, tnt, creeper-explosion)
- GET `/api/regions` returns `regions[]` with flag map from `region list json`

Chat, Guard, Protect, Map, World

These tabs **write plugin YAML** via `save-settings` (or equivalent) and reload the plugin — not just run one-off commands.

Network health (Dashboard tab)

`/api/status` includes `networkHealth`:

- Folia running, bedrock/crossplay/velocity flags
- Link process running (`linkProcessRunning`), config + suite completeness
- Plugin count + compat warning count
- **Ops plugins** — Phase 8 jar readiness (Protect, Chat, Moderation, Player data, Map, Discord, Tebex) with one-line detail per plugin

Map tab

Serves tiles via YaPcore pack HTTP when `use-yapcore-server: true` (default). First render runs ~2s after plugin enable; full re-render on `render-interval-minutes`. Tune `sample-chunk-radius` and `max-height` on low-CPU hosts — see plugin `config.yml` comments.

Wave 4 markers — live player markers via `/map/markers.json` (poll interval configurable). Optional NPC points and region outlines when YaPNpcs / YaPRegions are installed and toggled on in the Map tab. This is a **flat** Leaflet map; BlueMap-style **3D** mesh viewing is Stretch / out of scope until later.

Discord tab

Webhooks (moderation, chat, events), relay toggles, and join/leave/death/advancement event toggles. Safe setup order documented in [DISCORD_RELAY.md](#). MC→Discord and Discord→MC stay **off** until webhooks and inbound secrets are set.

YaP Link tab (proxy process)

Separate from the main **Console** tab (YaPcore/Folia). Requires `link-embed=false`.

Settings UI — configure the full proxy without editing files by hand:

- **Proxy settings** — bind, MOTD, max players, online mode, public host/port, chat relay, bedrock block
- **Backends** — add/remove servers (`hub` , `survival` , ...), host:port, optional per-backend Bedrock address
- **Try order** — fallback order when a backend is down (e.g. `hub` , `survival`)
- **Forced hosts** — route by hostname (e.g. `hub.yourdomain.com` → `hub`)
- **Hub / server selector** — `yaplink-server-selector` `hub` + session lock

Saves write `link-data/link.properties`. If Link is running, the dashboard sends `reload` automatically.

Action	POST /api/link body
Start Link	<code>{"action": "start"}</code>
Stop Link	<code>{"action": "stop"}</code>
Run command	<code>{"action": "command", "command": "reload"}</code>
Enable backend forwarding	<code>{"action": "enable-backend-forwarding"}</code>
Save proxy settings	<code>{"action": "save-proxy", ...}</code>
Save backends	<code>{"action": "save-servers", "servers": [...], "try": [...], "forcedHosts": [...]}</code>
Save selector	<code>{"action": "save-selector", "hubServer": "hub", "sessionLock": "true"}</code>

Live log: **GET** `/api/link/console` · **SSE** `/api/link/console/stream?token=...`

Plugin compat (Plugins tab)

Each jar shows status from [PLUGIN_COMPAT_MATRIX.md](#): `native` , `works` , `broken` , `folia-build` , or `unknown` , plus native alternative hint.

Ranks via console / tab

```
gradle installProductDefaults
# after Folia/Paper is up:
ranks apply
/yapperm user Steve parent set vip
/promote Steve
```

Dashboard **Access & ranks** and **Rank pack** tabs call `/api/access` and `/api/ranks`. Reference: [examples/yapperms/ranks-reference.txt](#). Config: `plugins/YaPPerms/config.yml`. See [PERMISSIONS.md](#).

Optional: `yap-ranks-auto-apply=true` in `config/server.properties`.

Security

- Treat the token like a password.
- Prefer `web-dashboard-localhost-only=true` or bind to a private IP, then put nginx + TLS in front for public access.
- Do not expose `:8080` to the internet without auth + TLS.
- Use **Network setup** → **Rotate token** if the secret may have leaked.